

1. Verdadero o Falso

1. Mientras más rápida sea su CPU y más RAM tenga un computador, menor será la complejidad de tiempo de un algoritmo que ejecuta.
2. Obtener el valor del elemento en la posición i de un arreglo de tamaño n tiene complejidad de tiempo del orden de n (lineal).
3. Siempre es posible eliminar el primer nodo de una lista enlazada con n elementos en tiempo constante.
4. Siempre es posible obtener el nodo en la posición i de una lista enlazada con n elementos en tiempo constante.
5. En una pila ("stack"), el último elemento que entra es el primero que sale.
6. Una fila virtual para comprar entradas a un concierto normalmente se gestiona con orden FIFO.
7. Las llamadas de una función autorecursiva se ejecutan en orden FIFO.
8. El siguiente método tiene complejidad de tiempo $\mathcal{O}(1)$.

```
public static int f1(int N) {  
    if (N == 0) return 0;  
    return f1(N - 1) + 1;  
}
```

9. El siguiente método devuelve el elemento más grande de una lista de n enteros positivos...

```
public static int grande(int n, List<Integer> elementos) {  
    int res = 0;  
    for (int i = 0; i < n; i++) {  
        res = Math.max(elementos.get(i), res);  
    }  
    return res;  
}
```

10. El output del siguiente método, cuando $N = 3$ y $M = 5$, es 2.

```
public static int mag(int N, int M) {  
    if (M == 0) return N;  
    return mag(M, N % M);  
}
```

11. La Búsqueda Binaria puede aplicarse sobre cualquier arreglo, independientemente de si sus elementos están ordenados o no, manteniendo una complejidad de $\mathcal{O}(\log n)$.
12. Una ventaja de las Listas Enlazadas sobre los Arreglos es que no requieren que la memoria esté asignada en bloques contiguos.
13. Si un algoritmo tiene complejidad temporal $\mathcal{O}(n)$ y otro $\mathcal{O}(2n)$, ambas complejidades tienen el mismo orden de crecimiento.
14. Una fila (cola) puede implementarse usando dos pilas.
15. Si $f(n) = \mathcal{O}(n^2)$ y $g(n) = \mathcal{O}(n \log n)$, entonces $f(n) + g(n) = \mathcal{O}(n^2 \log n)$.
16. Un aumento en la potencia de la CPU o de la memoria RAM mejora el rendimiento de un algoritmo en términos de su complejidad temporal asintótica Big-O.

17. Al usar una función autorrecursiva, las llamadas pendientes de terminar su ejecución se retoman en el mismo orden en que se hicieron.
18. Si una lista enlazada simple mantiene referencias tanto al primer nodo como al último, entonces eliminar el último nodo puede realizarse en tiempo constante.
19. Es posible aplicar el algoritmo de Búsqueda Binaria en una lista enlazada simple ordenada con tiempo de ejecución de $\mathcal{O}(\log n)$.
20. El acceso aleatorio permite acceder a cualquier elemento en tiempo $\mathcal{O}(1)$ en arreglos, mientras que en listas enlazadas el acceso por índice tiene tiempo de ejecución $\mathcal{O}(n)$.

2. Respuestas a Verdadero o Falso

1. **Falso.** La complejidad de tiempo asintótica (Big-O) es una medida teórica del número de operaciones en función del tamaño de entrada y es independiente del hardware.
2. **Falso.** El acceso a un índice en un arreglo toma tiempo $\mathcal{O}(1)$ porque la memoria es contigua.
3. **Verdadero.** El primer nodo (head) se puede eliminar en $\mathcal{O}(1)$ actualizando la referencia `head = head.next`.
4. **Falso.** Acceder a la posición i requiere recorrer la lista desde la cabeza nodo por nodo, tomando tiempo $\mathcal{O}(i)$ o $\mathcal{O}(n)$ en el peor caso.
5. **Verdadero.** Una pila funciona con la política LIFO (Last In, First Out).
6. **Verdadero.** Una fila funciona con la política FIFO (First In, First Out).
7. **Falso.** La pila de llamadas (Call Stack) funciona en orden LIFO.
8. **Falso.** El método hace N llamadas recursivas y cada una toma tiempo constante, resultando en complejidad $\mathcal{O}(N)$.
9. **Verdadero.** El método itera n veces comparando y actualizando el valor máximo.
10. **Falso.** Para $N = 3, M = 5$: $\text{mag}(3, 5) \rightarrow \text{mag}(5, 3) \rightarrow \text{mag}(3, 2) \rightarrow \text{mag}(2, 1) \rightarrow \text{mag}(1, 0) \rightarrow$ retorna 1, no 2.
11. **Falso.** La Búsqueda Binaria requiere estrictamente que los elementos estén ordenados.
12. **Verdadero.** Los nodos de una lista enlazada se reservan de forma dinámica y no requieren estar contiguos en memoria.
13. **Verdadero.** En notación Big-O las constantes multiplicativas se ignoran, por lo que ambas son $\mathcal{O}(n)$.
14. **Verdadero.** Es un problema clásico. Se usa una pila para encolar y otra para desencolar.
15. **Falso.** La complejidad de la suma está dominada por el término mayor. Así, $f(n) + g(n) = \mathcal{O}(n^2)$ ya que n^2 crece más rápido que $n \log n$.
16. **Falso.** El hardware afecta el tiempo real, pero no cambia el crecimiento asintótico de las operaciones del algoritmo (complejidad Big-O).
17. **Falso.** Al ser LIFO, las llamadas pendientes se retoman en orden inverso al que se hicieron.

18. **Falso.** Para eliminar el último nodo se requiere actualizar el penúltimo nodo para que apunte a `null`. Encontrar el penúltimo nodo requiere tiempo $\mathcal{O}(n)$.
19. **Falso.** En una lista enlazada no se puede acceder al elemento central en $\mathcal{O}(1)$, toma $\mathcal{O}(n)$, por lo que la búsqueda binaria terminaría tomando $\mathcal{O}(n)$.
20. **Verdadero.** Los arreglos ofrecen acceso aleatorio (Random Access) por índice en $\mathcal{O}(1)$, a diferencia de las listas.

3. Preguntas de Desarrollo

3.1. Primera y última posición

Considere un arreglo de números enteros ordenado de manera ascendente. El arreglo puede contener elementos repetidos. Diseñe un algoritmo que haga uso de la búsqueda binaria y, dado el valor de un elemento buscado, imprima la primera y última posición en la que aparece (las posiciones en el arreglo parten en 0). Dicho algoritmo debe recibir como entrada un arreglo de enteros y el elemento a buscar. Si el elemento no está presente, debe imprimir -1.

- (1) Determine output para: `Array [ 5, 8, 9, 15, 15, 15, 17]`, `key = 15`
- (2) Determine output para: `Array [ 5, 8, 9, 15, 15, 15, 17]`, `key = 14`
- (3) Diseñe algoritmo.
- (4) Implemente en Java usando `binarySearch` de la librería estándar.

3.2. Compilador A Papel

Considere algoritmos sobre listas enlazadas. Se dan funciones `f1` y `f2` para analizar paso a paso, dibujar las listas resultantes y explicar qué hace cada método y su complejidad temporal. (Sin implementación en esta sección).

3.3. Conjetura de Collatz

Dado un entero n , si n es par, se divide por 2. Si es impar, se multiplica por 3 y se suma 1. Se repite hasta que $n = 1$. Se define la longitud de la secuencia de n . Dados i, j , determinar el número n en el rango que genere la mayor secuencia.

- (1) Imprimir secuencia de $n = 53$.
- (2) Implementar algoritmo recursivo que imprima la secuencia.
- (3) Diseñar algoritmo con memoización y implementarlo en Java.

3.4. Paréntesis Balanceado

Considere cadenas compuestas por corchetes, paréntesis y llaves. El algoritmo debe retornar un booleano indicando si la expresión es válida.

- (1) Verificar expresión a mano.
- (2) Diseñar algoritmo.
- (3) Implementar en Java.

3.5. Notación de Postfijo

Diseñe e implemente en Java un algoritmo que recibe una expresión aritmética de postfijo como un texto y calcula el resultado de evaluarla. El método debe recibir un `String` y devolver un `int`. Usar solo suma, resta, multiplicación y división.

3.6. Suma de Listas

Considere el uso de listas enlazadas para representar números enteros: con un dígito por nodo, en orden inverso. Por ejemplo $2 \rightarrow 9 \rightarrow \text{null}$ representa 92. Defina un método `suma` que recibe dos `head` y retorna el `head` de la lista resultante de la suma. Analice complejidad.

3.7. Algoritmo Misterioso (Listas)

Se presenta una función `MysteriousAlgorithm` recursiva que recibe el `head` de una lista. Se pide:

- (a) Describir resultado devuelto.
- (b) Explicar qué hace.
- (c) Analizar complejidad.

3.8. Torre de Minecraft

Jugadores construyen una torre de n peldaños. Se puede subir de 1 o de 2 peldaños a la vez. $C(n)$ cantidad de formas.

- (1) Relación de recurrencia.
- (2) Implementar en Java un método ineficiente.
- (3) Explicar por qué es malo para $n = 10^6$.
- (4) Solución eficiente en Java.

3.9. Algoritmo Misterioso (StringBuilder)

Se analiza un algoritmo sobre una lista enlazada de caracteres. Cuenta caracteres repetidos y los añade en un `StringBuilder`. Se pide trazar el resultado y hallar complejidad.

3.10. Playlist

Diseñar clase `Reproductor` con métodos `reproducir(id)`, `anterior()`, `siguiente()`. Usando dos pilas (`back` y `forward`). Implemente en Java.

4. Respuestas (Código en Java)

4.1. Primera y última posición

```
import java.util.Arrays;

public class BusquedaPosiciones {
    public static void buscarPosiciones(int[] arr, int key) {
        int index = Arrays.binarySearch(arr, key);
        if (index < 0) {
            System.out.println("-1");
            return;
        }
        int first = index;
        int last = index;

        while (first > 0 && arr[first - 1] == key) {
            first--;
        }
        while (last < arr.length - 1 && arr[last + 1] == key) {
            last++;
        }
        System.out.println(first + ", " + last);
    }
}
```

```
}
```

4.2. Conjetura de Collatz

```
import java.util.HashMap;

public class Collatz {
    // 1. Algoritmo recursivo basico
    public static void imprimirSecuencia(long n) {
        System.out.print(n + " ");
        if (n == 1) return;
        if (n % 2 == 0) imprimirSecuencia(n / 2);
        else imprimirSecuencia(3 * n + 1);
    }

    // 2. Optimizacion con memoizacion
    private static HashMap<Long, Long> memo = new HashMap<>();

    public static long longitudCollatz(long n) {
        if (n == 1) return 1;
        if (memo.containsKey(n)) return memo.get(n);

        long len;
        if (n % 2 == 0) {
            len = 1 + longitudCollatz(n / 2);
        } else {
            len = 1 + longitudCollatz(3 * n + 1);
        }
        memo.put(n, len);
        return len;
    }

    public static void mayorSecuencia(int i, int j) {
        long maxLen = 0;
        int maxNum = i;
        for (int num = i; num <= j; num++) {
            long len = longitudCollatz(num);
            if (len > maxLen) {
                maxLen = len;
                maxNum = num;
            }
        }
        System.out.println(maxNum + " " + maxLen);
    }
}
```

4.3. Paréntesis Balanceado

```
import java.util.Stack;

public class Parentesis {
    public static boolean esValida(String exp) {
        Stack<Character> stack = new Stack<>();
        for (char c : exp.toCharArray()) {
            if (c == '(' || c == '[' || c == '{') {
                stack.push(c);
            } else if (c == ')' || c == ']' || c == '}') {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if ((c == ')') && top != '(' ||
```

```

        (c == ']' && top != '[') ||
        (c == '}' && top != '{')) {
            return false;
        }
    }
}
return stack.isEmpty();
}
}

```

4.4. Notación de Postfijo

```

import java.util.Stack;

public class Postfijo {
    public static int evaluar(String expresion) {
        Stack<Integer> stack = new Stack<>();
        String[] tokens = expresion.split(" ");
        for (String token : tokens) {
            if (token.equals("+") || token.equals("-") || token.equals("*") ||
                token.equals("/")) {
                int b = stack.pop();
                int a = stack.pop();
                switch (token) {
                    case "+": stack.push(a + b); break;
                    case "-": stack.push(a - b); break;
                    case "*": stack.push(a * b); break;
                    case "/": stack.push(a / b); break;
                }
            } else {
                stack.push(Integer.parseInt(token));
            }
        }
        return stack.pop();
    }
}

```

4.5. Suma de Listas

```

class LinkedList {
    class Node {
        int value;
        Node next;
        Node(int v) { value = v; }
    }
    Node head;

    public Node suma(Node head1, Node head2) {
        Node dummy = new Node(0);
        Node current = dummy;
        int carry = 0;

        while (head1 != null || head2 != null || carry > 0) {
            int sum = carry;
            if (head1 != null) {
                sum += head1.value;
                head1 = head1.next;
            }
            if (head2 != null) {
                sum += head2.value;
            }
            current.next = new Node(sum % 10);
            current = current.next;
            carry = sum / 10;
        }
        return dummy.next;
    }
}

```

```

        head2 = head2.next;
    }
    carry = sum / 10;
    current.next = new Node(sum % 10);
    current = current.next;
}
return dummy.next;
}
}

```

4.6. Torre de Minecraft

```

public class TorreMinecraft {
    // Solucion ineficiente (recursiva)
    public static long formasDeSubirIneficiente(int n) {
        if (n == 0 || n == 1) return 1;
        return formasDeSubirIneficiente(n - 1) + formasDeSubirIneficiente(n - 2);
    }

    // Solucion eficiente (Programacion dinamica)
    public static long formasDeSubir(int n) {
        if (n == 0 || n == 1) return 1;
        long prev2 = 1;
        long prev1 = 1;
        long current = 0;
        for (int i = 2; i <= n; i++) {
            current = prev1 + prev2;
            prev2 = prev1;
            prev1 = current;
        }
        return current;
    }
}

```

4.7. Playlist

```

import java.util.Stack;

public class Reproductor {
    private Stack<String> back;
    private Stack<String> forward;
    private String actual;

    public Reproductor() {
        back = new Stack<>();
        forward = new Stack<>();
        actual = null;
    }

    public void reproducir(String id) {
        if (actual != null) {
            back.push(actual);
        }
        actual = id;
        forward.clear();
    }

    public void anterior() {
        if (!back.isEmpty()) {

```

```
        forward.push(actual);
        actual = back.pop();
    }
}

public void siguiente() {
    if (!forward.isEmpty()) {
        back.push(actual);
        actual = forward.pop();
    }
}
}
```